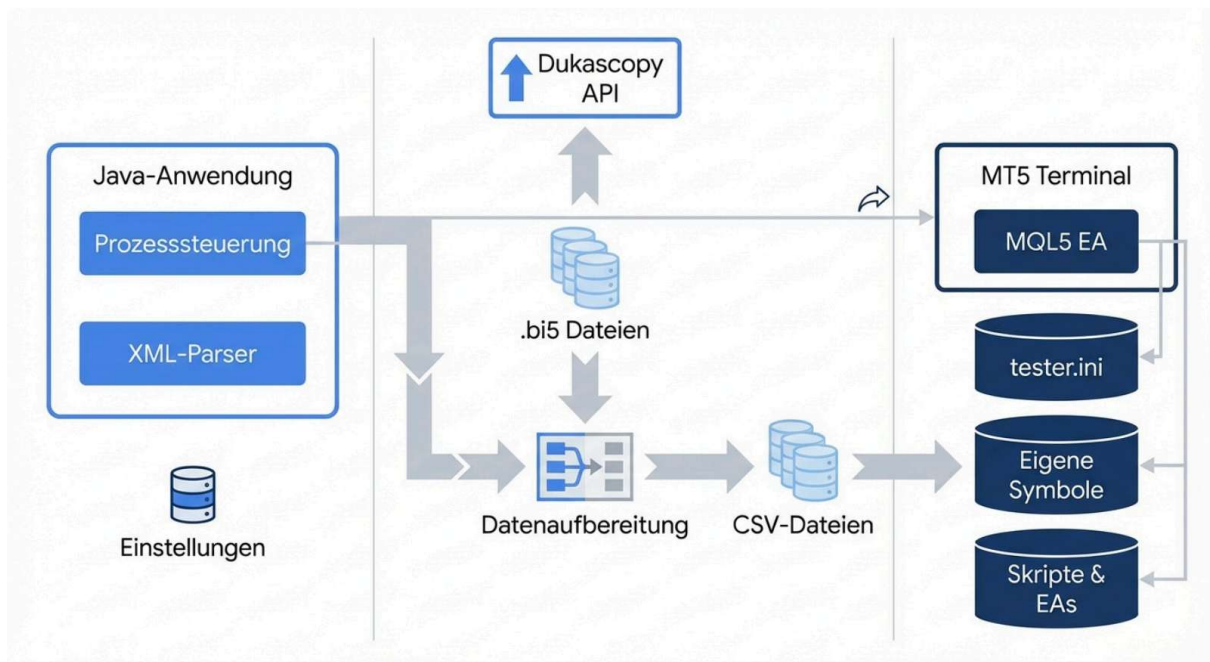


Technische Spezifikation und Architektur-Referenz: Automatisierung von MetaTrader 5 Backtests mittels Java-Systemintegration

Die vorliegende architektonische Ausarbeitung dient als umfassende Wissensbasis und technische Spezifikation für die Entwicklung einer Java-basierten Desktop-Applikation (GUI). Ziel der Software ist die vollautomatisierte Orchestrierung des MetaTrader 5 (MT5) Strategietesters, die Verwaltung historischer Kursdaten sowie die Auswertung von Backtest-Ergebnissen. Die Analyse evaluiert die Schnittstellen der MT5-Kommandozeile, den Import von hochauflösenden Dukascopy-Daten via Custom Symbols und die notwendigen Java-Bibliotheken für ein thread-sicheres Prozess-Management.

Systemarchitektur und asynchroner Kontrollfluss



Der Kontrollfluss initiiert asynchrone Daten-Downloads, transformiert diese in CSV-Dateien und injiziert sie via MQL5-Skript in MetaTrader 5, bevor der Java ProcessBuilder den eigentlichen Backtest über die CLI startet.

1. Schnittstellen und CLI-Steuerung von MetaTrader 5

MetaTrader 5 ist primär als monolithische Desktop-Anwendung konzipiert, bietet jedoch eine native Kommandozeilenschnittstelle (Command Line Interface, CLI), um das Terminal sowie den integrierten Strategietester programmgesteuert zu automatisieren. Für ein externes Java-Programm stellt diese CLI den stabilsten und deterministischsten Weg dar, um Backtests "headless" oder in einem vollautomatisierten Workflow anzustoßen. Die Ausführung stützt sich auf Konfigurationsdateien, welche die manuellen Eingaben des Benutzers in der grafischen Oberfläche des Strategietesters vollständig ersetzen.

1.1 Der Aufruf über die Kommandozeile (terminal64.exe)

Um MT5 über die CLI zu steuern, wird die ausführbare Datei terminal64.exe aufgerufen. Der essenzielle Parameter für das automatisierte Testing ist /config, welcher den absoluten oder relativen Pfad zu einer Konfigurationsdatei übermittelt, die standardmäßig die Dateiendung .ini trägt. Diese Konfigurationsdateien werden vom Terminal im Nur-Lese-Modus ("read only") verarbeitet; Änderungen, die während der Laufzeit in der Plattform vorgenommen werden, überschreiben diese Datei nicht.

Darüber hinaus ist der Startparameter /portable von absolut kritischer Bedeutung für die Systemintegration.¹ Ohne diesen Parameter speichert MetaTrader 5 Konfigurationsdaten, Logs, historische Kursdaten und Testergebnisse standardmäßig im versteckten Windows-Benutzerverzeichnis

(C:\Users\<User>\AppData\Roaming\MetaQuotes\Terminal\<Hash>\).¹ Dieses Verhalten erschwert die programmatische Lokalisierung der generierten Report-Dateien durch die Java-Applikation extrem, da der Hash-Wert des Verzeichnisses dynamisch generiert wird. Der Portable-Modus erzwingt, dass das Terminal das eigene Installationsverzeichnis als Root-Verzeichnis für sämtliche Lese- und Schreiboperationen verwendet. Voraussetzung hierfür ist, dass der Windows-Benutzer oder der ausführende Java-Prozess über ausreichende Schreibrechte in diesem Verzeichnis verfügt oder das Terminal außerhalb des restriktiven C:\Program Files\ Verzeichnisses installiert wird.

Ein syntaktisch korrekter Kommandozeilenaufruf aus der Java-Umgebung lautet somit ⁴:

DOS

```
"C:\Trading\MetaTrader5\terminal64.exe" /portable /config:C:\Trading\Jobs\tester_001.ini
```

1.2 Aufbau der tester.ini Konfigurationsdatei

Die INI-Datei fungiert beim Start als präzise Spezifikation für den Backtest. Sie überschreibt temporär die GUI-Einstellungen des Terminals. Für das Java-Programm ist es die primäre

Aufgabe, diese Datei dynamisch für jeden Backtest-Durchlauf neu zu generieren. Die Datei ist in verschiedene Blöcke unterteilt, wobei für den Backtest der Block `` zwingend erforderlich ist.¹

Die Konfigurationsebene unterstützt eine Vielzahl von Parametern, die das exakte Verhalten der Testing-Engine determinieren. Die folgende Tabelle spezifiziert die wichtigsten Parameter und ihre Wertebereiche, die das Java-Programm generieren muss¹:

Parameter	Datentyp	Beschreibung und Wertebereich
Expert	String	Der Dateiname des Expert Advisors (EA), der getestet werden soll. Die Angabe erfolgt ohne die .ex5 Dateiendung und relativ zum Verzeichnis MQL5\Experts\. Fehlt dieser Parameter, wird der Tester nicht gestartet.
ExpertParameters	String	Der Name der Datei (z.B. .set), die die Input-Parameter des EAs enthält. Diese muss im Verzeichnis MQL5\Profiles\Tester liegen.
Symbol	String	Das primäre Finanzinstrument für den Test (z.B. EURUSD oder ein Custom Symbol wie EURUSD_Duka).
Period	String	Die Zeiteinheit des Charts. Erlaubt sind die 21 MT5-Perioden (z.B. M1, M5, H1, D1). Standard ist H1.
Model	Integer	Das Modell der

		Tick-Generierung. 0 = "Every tick" (Jeder Tick), 1 = "1 minute OHLC", 2 = "Open price only", 3 = "Math calculations", 4 = "Every tick based on real ticks" (Reale Ticks).
ExecutionMode	Integer	Emulation der Handelsausführung. 0 = Normal, -1 = Zufällige Verzögerung, >0 = Verzögerung in Millisekunden (max. 600.000).
FromDate	Date	Startdatum des Tests im Format YYYY.MM.DD.
ToDate	Date	Enddatum des Tests im Format YYYY.MM.DD.
Deposit	Integer	Die initiale Kontogröße für den simulierten Backtest (z.B. 10000). ⁶
Currency	String	Die Kontowährung (z.B. USD oder EUR). ¹
Leverage	String	Der simulierte Hebel des Kontos im Format 1:100. ¹
Optimization	Integer	Aktiviert die Optimierung. 0 = Deaktiviert (Einzeltest), 1 = Langsamer kompletter Algorithmus, 2 = Schneller genetischer Algorithmus.
OptimizationCriterion	Integer	Das Zielkriterium für den genetischen Algorithmus. 0 = Max. Balance, 1 = Balance

		+ Profitabilität, 2 = Balance + Expected Payoff, 3 = (100% - Drawdown)*Balance, 4 = Balance + Recovery Factor.
Report	String	Der Name der Ausgabedatei. Relativ zum Plattform-Verzeichnis. Für Einzeltests wird standardmäßig .htm generiert. Um eine programmatisch verarbeitbare Datei zu erhalten, muss .xml erzwungen werden oder die Datei wird im Nachgang in Java geparkt.
ReplaceReport	Integer	Erlaubt das Überschreiben existierender Berichte. 1 = Aktiviert, 0 = Deaktiviert. Ist dies deaktiviert, hängt MT5 bei Namenskonflikten eine Zahl in eckigen Klammern an (z.B. tester.htm).
ShutdownTerminal	Integer	Beendet das Terminal nach Abschluss des Tests. 1 = Aktiviert, 0 = Deaktiviert. Essenziell für externe Aufrufe.

Eine vollständige, produktionsreife tester.ini, die durch die Java-Software erzeugt wird, stellt sich wie folgt dar ¹:

Ini, TOML

Expert=MyCustomEAs\MovingAverageCross

```

Symbol=EURUSD_Duka
Period=H1
Model=4
ExecutionMode=0
FromDate=2023.01.01
ToDate=2023.12.31
Deposit=10000
Currency=USD
Leverage=1:100
Optimization=0
Report=Reports\Backtest_EURUSD_H1.xml
ReplaceReport=1
ShutdownTerminal=1

```

Besonderes Augenmerk liegt auf dem Parameter ShutdownTerminal=1. Ohne diesen Parameter verbleibt der MT5-Prozess nach dem erfolgreichen Abschluss des Backtests unendlich lange im Arbeitsspeicher des Systems. Durch den Wert 1 signalisiert das Terminal dem Betriebssystem (und damit der aufrufenden Java-Applikation) einen regulären Exit-Code (0), sobald der Report generiert wurde. Weiterhin bestimmt der Parameter Report das Ausgabeformat. Die Spezifikation der Dateiendung .xml im Parameter Report zielt darauf ab, ein maschinenlesbares Format zu generieren, was für die spätere Verarbeitung in Java essenziell ist. Wenn der Tester in bestimmten Versionen hartnäckig HTML für Einzeltests generiert, muss die Java-Software so konzipiert sein, dass sie auch strukturierte HTML-Tabellen parsen kann.⁶

1.3 Erkennung der Backtest-Beendigung in Java

Da die Java-Applikation den MT5-Prozess via `java.lang.ProcessBuilder` startet, stehen zwei primäre Architekturansätze zur Verfügung, um das Ende des Backtests zu registrieren und den Auswertungsprozess (das Parsen der Dateien) zu initiieren.

Ansatz A: Synchrone Prozess-Überwachung (Empfohlen) Da ShutdownTerminal=1 in der INI-Datei gesetzt ist, beendet sich das Programm selbstständig. In Java wird der ausführende Task-Thread mittels der Methode `process.waitFor()` blockiert, bis der Exit-Code des Betriebssystems zurückgeliefert wird.⁸ Dies ist der robusteste und sicherste Weg. Die Rückgabe von `waitFor()` garantiert auf Betriebssystemebene, dass der Kindprozess (`terminal64.exe`) vollständig beendet wurde, alle Datei-Handles des Terminals geschlossen wurden und die Ausgabedateien (der Report) vollständig auf die Festplatte geschrieben und synchronisiert (flushed) wurden.⁸ Erst nach der Rückkehr von `waitFor()` initiiert Java das Parsen der XML/HTML-Datei.

Ansatz B: Asynchrone Dateisystem-Überwachung (WatchService) Falls das Terminal aus architektonischen Gründen geöffnet bleiben muss (beispielsweise bei aufeinanderfolgenden Tests ohne den Overhead eines kompletten Neustarts), scheidet `waitFor()` aus. In diesem Fall

kann die native Java API `java.nio.file.WatchService` verwendet werden.¹⁰ Diese registriert asynchrone Events wie `ENTRY_CREATE` oder `ENTRY_MODIFY` in dem spezifischen Reports-Verzeichnis, welches in der INI-Datei definiert wurde.¹² Die erhebliche Herausforderung hierbei besteht in potenziellen Datei-Sperren (File Locks). Es besteht die Gefahr, dass der MT5-Prozess noch sukzessive in die Report-Datei schreibt, während der `WatchService` das Event bereits feuert und Java versucht, die Datei zu lesen.¹¹ Dies führt zu `IOExceptions` wegen Dateizugriffskonflikten. Ein Workaround ist das Warten auf die Freigabe des File-Locks durch wiederholtes Pollen. Ansatz A ist jedoch aufgrund seiner deterministischen Natur in Automatisierungs-Pipelines strikt vorzuziehen.

1.4 Evaluierung: Java Wrapper vs. Python MetaTrader5 Bibliothek

Eine alternative Architektur zur Prozess-Steuerung stellt die Nutzung der offiziellen Python-Bibliothek `MetaTrader5` dar.¹⁴ `MetaQuotes` bietet diese Bibliothek an, um direkt mit dem Terminal via Interprozesskommunikation (IPC) zu interagieren.¹⁴ Dies ermöglicht den performanten Datenabruf, das Ausführen von Live-Trades und das Abfragen von Kontoinformationen.¹⁴ Es stellt sich die architektonische Frage, ob die Java-GUI lediglich als Frontend fungieren und Python-Skripte für die MT5-Kommunikation aufrufen sollte.

Die tiefergehende Analyse der offiziellen MQL5-Python-Dokumentation ergibt jedoch einen kritischen Engpass für dieses Vorhaben: Die Python-Bibliothek besitzt **keine** nativen Funktionen zum Starten, Steuern oder Auslesen des Strategietesters.¹⁶ Funktionen wie `mt5.initialize()` starten zwar das Terminal im Hintergrund, und Funktionen wie `copy_rates_from` oder `order_send` erlauben die Interaktion, doch existiert keine API-Methode wie `mt5.run_backtest()`.¹⁴

Frameworks von Drittanbietern, wie beispielsweise `StrategyTester5` für Python, umgehen dieses fundamentale Fehlen, indem sie eine eigene Backtest-Engine in Python nachbauen.¹⁶ Diese Frameworks fordern historische Daten vom MT5-Terminal an und simulieren die Trade-Ausführung vollständig in Python.¹⁶ Dies garantiert jedoch nicht hundertprozentig dieselben Ausführungslogiken, Slippage-Modelle und Margin-Berechnungen wie der native, in C++ programmierte MT5-Strategietester.

Architektonische Entscheidung: Da das erklärte Ziel der Software die Nutzung des nativen, hochgradig optimierten MT5-Backtesters ist, bietet ein Python-Wrapper für diesen spezifischen Anwendungsfall keinen systemischen Vorteil.⁵ Die Java-Anwendung sollte auf Python-Skripte für die *Steuerung des Testers* verzichten und die INI-Erstellung sowie den Aufruf der `terminal64.exe` via `ProcessBuilder` direkt selbst in Java implementieren. Dies reduziert die Komplexität und schließt Fehlerquellen aus, die durch fehlerhafte Python-Environment-Pfade oder IPC-Timeouts entstehen könnten.

2. Beschaffung und Import von Dukascopy-Kursdaten

Für professionelle, realitätsnahe Backtests ist die Qualität der vom Broker standardmäßig

gelieferten Kurshistorie oft unzureichend. Lückenhafte Daten oder interpolierte Tick-Volumina verfälschen die Ergebnisse, insbesondere bei Scalping-Strategien. Der Schweizer Broker Dukascopy bietet eine frei zugängliche, hochauflösende Datenbank mit extrem präzisen Tick-Daten (OHLCV). Diese in MetaTrader 5 nutzbar zu machen, erfordert einen mehrstufigen Transformationsprozess.

2.1 Struktur des Dukascopy.bi5-Formats

Dukascopy speichert und distribuiert seine Tick-Daten nicht in lesbaren Textformaten wie CSV, sondern in proprietären .bi5 Dateien. Dabei handelt es sich um binäre Dateien, die zur Reduktion der Bandbreite mit dem LZMA-Algorithmus (Lempel-Ziv-Markov chain-Algorithm) komprimiert sind.¹⁹ Jede dieser Dateien deckt typischerweise eine Handelsstunde für ein spezifisches Instrument ab.

Entpackt man die .bi5 Datei, offenbart sich eine stark optimierte, sequenzielle Struktur von 20-Byte-Blöcken.²¹ Jeder 20-Byte-Block repräsentiert einen einzigen Tick auf dem Markt und besteht aus fünf 32-Bit (4-Byte) Werten, die im Big-Endian-Format encodiert sind²¹:

1. TIME (4 Bytes): Ein 32-Bit Integer, der die verstrichenen Millisekunden seit dem Beginn der jeweiligen Stunde (der Epoche der Datei) repräsentiert.²¹
2. ASKP (4 Bytes): Der Ask-Preis. Um Speicherplatz zu sparen, wird dieser als 32-Bit Integer gespeichert, der den realen Preis multipliziert mit einem symbolspezifischen Faktor (oft 100.000 bei Währungspaaren) darstellt.²¹
3. BIDP (4 Bytes): Der Bid-Preis, analog zum Ask-Preis als skaliertes Integer gespeichert.²¹
4. ASKV (4 Bytes): Das Ask-Volumen. Dies ist ein 32-Bit Float, der das Volumen dividiert durch 1.000.000 repräsentiert.²¹
5. BIDV (4 Bytes): Das Bid-Volumen, analog zum Ask-Volumen.²¹

Das Dekodieren erfordert somit das Einlesen der Bytes, die Konvertierung von Big-Endian in die systemlokale Endianness und die Anwendung der Divisoren, um die realen Gleitkommawerte für Preis und Volumen zu erhalten.²¹

2.2 Automatisierung des Downloads und der Dekompression

Um die enormen Datenmengen für langjährige Backtests zu bewältigen, ist eine Automatisierung des Downloads und der Konvertierung zwingend. Hierbei existieren zwei primäre architektonische Wege für das Java-System.

Weg 1: Orchestrierung von Python-Skripten Im Open-Source-Bereich hat sich Python als äußerst robust für diese Aufgabe erwiesen. Bibliotheken wie dukascopy-python²⁴ oder das CLI-Tool duka²⁵ können historische Daten über Jahre hinweg parallel herunterladen, dekomprimieren und direkt als Pandas DataFrame verarbeiten und als CSV speichern.²⁴ Das Java-System fungiert hier als Orchestrator, der via ProcessBuilder einen Python-Prozess aufruft.

Ein Minimalbeispiel basierend auf dukascopy-python, welches minütliche Daten abruft ²⁴:

Python

```
from datetime import datetime
import dukascopy_python
import sys

def download_data(symbol, start_str, end_str, output_file):
    start = datetime.strptime(start_str, "%Y-%m-%d")
    end = datetime.strptime(end_str, "%Y-%m-%d")

    # Abruf hochauflösender M1-Daten
    interval = dukascopy_python.INTERVAL_MINUTE_1
    offer_side = dukascopy_python.OFFER_SIDE_BID

    df = dukascopy_python.fetch(
        symbol, interval, offer_side, start, end,
    )

    # Speichern als MT5-kompatible CSV
    df.to_csv(output_file, index=False, header=True)

if __name__ == "__main__":
    download_data(sys.argv, sys.argv, sys.argv, sys.argv)
```

Weg 2: Native Java-Implementierung Sollte die Abhängigkeit von einer Python-Umgebung auf dem Zielsystem unerwünscht sein, kann die Logik nativ in Java abgebildet werden. Der Download erfolgt über asynchrone HTTP-Requests (z.B. via `java.net.http.HttpClient`), wobei die URL-Struktur von Dukascopy prädiktiv ist (z.B. `Symbol/Year/Month/Day/Hourh_ticks.bi5`).²³ Für die LZMA-Dekompression stellt die Open-Source-Bibliothek `org.tukaani.xz` die Klasse `LZMAInputStream` zur Verfügung.²⁶ Die Java-Implementierung liest den Stream sequenziell und wendet die Klasse `java.nio.ByteBuffer` an, um die 20-Byte-Blöcke zu parsen.

Java

```
import org.tukaani.xz.LZMAInputStream;
```

```

import java.io.*;
import java.nio.ByteBuffer;

public class Bi5Parser {
    public void parseBi5(String inputPath, String outputPath) throws Exception {
        try (InputStream fileIn = new FileInputStream(inputPath);
            InputStream lzmaIn = new LZMAInputStream(fileIn);
            BufferedWriter writer = new BufferedWriter(new FileWriter(outputPath))) {

            byte buffer = new byte[20]; // 20 Bytes pro Tick
            while (lzmaIn.read(buffer) == 20) {
                ByteBuffer bb = ByteBuffer.wrap(buffer);
                int timeOffset = bb.getInt();
                int ask = bb.getInt();
                int bid = bb.getInt();
                float askVol = bb.getFloat();
                float bidVol = bb.getFloat();

                // Konvertierungslogik anwenden (z.B. Division durch 100.000)
                // In CSV schreiben...
            }
        }
    }
}

```

2.3 Das Architektur-Problem der Zeitzone-Konvertierung

Eine der kritischsten Hürden bei der Verwendung von Fremddaten ist das Zeitzone-Management. Dukascopy liefert alle historischen Daten strikt in der Zeitzone UTC (Coordinated Universal Time, GMT+0).²⁰ Der Devisenmarkt (Forex) und infolgedessen die Plattform Metatrader 5 basieren in der Regel jedoch auf der "New York Close" Methodik. Das bedeutet, ein globaler Handelstag endet formell um 17:00 Uhr New Yorker Zeit (EST/EDT).²⁷

Die überwältigende Mehrheit der MT5-Broker – und damit auch die darauf abgestimmten historischen EA-Logiken – betreiben ihre Server auf der Zeitzone GMT+2 während der Standardzeit und GMT+3 (Eastern European Time, EET) während der Sommerzeit (Daylight Saving Time, DST).²⁷

Dieses Setup garantiert, dass 00:00 Uhr Serverzeit exakt mit 17:00 Uhr New Yorker Zeit übereinstimmt. Das Resultat ist eine Handelswoche mit exakt fünf täglichen Kerzen (Montag bis Freitag).²⁷ Wenn nun UTC-Daten von Dukascopy unbearbeitet in MT5 geladen werden, verschiebt sich der Tageswechsel. Es entstehen unweigerlich verkürzte "Sonntags-Kerzen" auf dem Tageschart.²⁷ Dies hat katastrophale Auswirkungen auf die Berechnungen von technischen Indikatoren wie Gleitenden Durchschnitten (Moving Averages) oder Oszillatoren,

da diese durch die zusätzlichen, volatilitätsarmen Sonntags-Datenpunkte stark verfälscht werden.²⁷

Die Daten-Pipeline der Java-Software muss dieses Problem lösen, bevorzugt direkt nach der LZMA-Dekompression. Die Zeitstempel jedes Ticks oder jeder OHLC-Kerze müssen iteriert und dynamisch anhand der historischen US/EU-Sommerzeitwechsel von UTC auf GMT+2/GMT+3 verschoben werden.²⁷ Die Implementierung muss historische DST-Regeln präzise abbilden (z.B. den Wechsel am letzten Sonntag im März und letzten Sonntag im Oktober für Europa)²⁷, bevor die resultierende CSV-Datei in MT5 eingespeist wird.

2.4 Import in MetaTrader 5 über Custom Symbols

MetaTrader 5 ist restriktiv konzipiert und erlaubt keine direkte Modifikation oder das Überschreiben der Standard-Historien (z.B. des vom Broker gelieferten Symbols EURUSD).³² Um fremde Daten in den Strategietester zu speisen, muss ein sogenanntes "Custom Symbol" (Benutzerdefiniertes Symbol) erstellt werden.³³ Diese virtuellen Instrumente verhalten sich exakt wie reguläre Symbole, jedoch hat der Anwender die volle Kontrolle über deren Preisdaten und Spezifikationen.³⁴

Die Java-Software kann den Import theoretisch durch manuelle Interaktion des Users in der GUI (View -> Symbols -> Custom Symbol -> Import Bars) anleiten.³² Dies ist jedoch fehleranfällig und widerspricht dem Prinzip der Vollautomatisierung. Der korrekte architektonische Ansatz ist das Bereitstellen eines automatisierten MQL5-Skripts, welches die aufbereiteten CSV-Dateien aus dem Java-Prozess ausliest und programmatisch injiziert.³⁴

Damit der manuelle oder programmatische Import reibungslos funktioniert, muss die erzeugte CSV-Datei strikt formatiert sein. Das von MetaTrader 5 geforderte Format umfasst acht spezifische Spalten in genauer Reihenfolge: Date Time Open High Low Close TickVolume Volume Spread oder alternativ eine kombinierte DateTime-Spalte.³⁶ Fehlen Spalten, markiert der Tester die Daten beim manuellen Import rot (als fehlerhaft).³⁷

2.5 MQL5-Skript zur automatisierten Dateninjektion

Um die Lücke zwischen der Java-CSV-Erzeugung und der MT5-Datenbank zu schließen, wird ein MQL5-Skript (.mq5) geschrieben und kompiliert. Die MQL5-API stellt hierfür dedizierte Funktionen wie CustomSymbolCreate, CustomSymbolSetInteger und CustomRatesReplace zur Verfügung.³⁸ Die Java-GUI verschiebt die fertige CSV in das Verzeichnis MQL5\Files des MT5-Terminals und startet das Skript über die Kommandozeile.

Das folgende MQL5-Referenz-Skript demonstriert den vollautomatisierten Importprozess³³:

Code-Snippet

```
//+-----+
//| Custom Symbol Importer (MQL5) |
//+-----+
#property script_show_inputs

// Input-Parameter können von außen (z.B. via INI) gesetzt werden
input string  InpFileName  = "Dukascopy_EURUSD_M1.csv";
input string  InpCustomName = "EURUSD_Duka";
input string  InpOriginName = "EURUSD"; // Original-Symbol für Spezifikationen

void OnStart()
{
    // 1. Custom Symbol erstellen (kopiert Spezifikationen des Originals)
    if(!SymbolInfoInteger(InpCustomName, SYMBOL_CUSTOM))
    {
        if(CustomSymbolCreate(InpCustomName, "CustomData", InpOriginName))
        {
            Print("Custom Symbol ", InpCustomName, " erfolgreich erstellt.");

            // Konfiguration: Chart-Darstellung erzwingen
            // Wichtig: SYMBOL_CHART_MODE definiert, ob Bars nach Bid oder Last gezeichnet
            // werden.
            CustomSymbolSetInteger(InpCustomName, SYMBOL_CHART_MODE,
            SYMBOL_CHART_MODE_BID);

            // Ausführungsmodus für Backtests auf Market Execution setzen
            CustomSymbolSetInteger(InpCustomName, SYMBOL_TRADE_EXEMODE,
            SYMBOL_TRADE_EXECUTION_MARKET);
        }
        else
        {
            Print("Fehler bei Erstellung: ", GetLastError());
            return;
        }
    }

    // 2. CSV Datei öffnen
    int fileHandle = FileOpen(InpFileName, FILE_READ|FILE_CSV|FILE_ANSI, ',');
    if(fileHandle == INVALID_HANDLE)
    {
        Print("CSV Datei konnte nicht geöffnet werden.");
        return;
    }
}
```

```
}

FileReadString(fileHandle); // Header-Zeile überspringen

// 3. MqlRates Array befüllen
MqlRates rates;
int count = 0;

// Performance-Optimierung: Speicher vorab allokieren, um Re-Allocations in der Schleife zu
minimieren
ArrayResize(rates, 500000);

while(!FileIsEnding(fileHandle))
{
    if(count >= ArraySize(rates)) ArrayResize(rates, count + 100000);

    string timeStr = FileReadString(fileHandle);
    rates[count].time = StringToTime(timeStr);
    rates[count].open = FileReadNumber(fileHandle);
    rates[count].high = FileReadNumber(fileHandle);
    rates[count].low = FileReadNumber(fileHandle);
    rates[count].close = FileReadNumber(fileHandle);
    rates[count].tick_volume = (long)FileReadNumber(fileHandle);
    rates[count].spread = 0;
    rates[count].real_volume = 0;

    count++;
}
FileClose(fileHandle);

// Array auf tatsächliche Größe schrumpfen
ArrayResize(rates, count);

// 4. Daten injizieren
if(count > 0)
{
    // CustomRatesReplace löscht alte Historie in der Range und ersetzt sie komplett
    int replaced = CustomRatesReplace(InpCustomName, rates.time, rates[count-1].time,
rates);
    if(replaced < 0) {
        Print("Fehler bei CustomRatesReplace: ", GetLastError());
    } else {
        Print("Erfolgreich injiziert: ", replaced, " Bars.");
    }
}
```

```
}  
  
    // Symbol im Market Watch aktivieren, damit der Tester es findet  
    SymbolSelect(InpCustomName, true);  
}  
}
```

Die Funktion CustomRatesReplace ist hierbei der ebenfalls existierenden Methode CustomRatesUpdate konzeptionell vorzuziehen.³⁸ Während Update lediglich neue Bars hinzufügt und bestehende unangetastet lässt, überschreibt Replace die Historie im definierten Zeitraum vollständig.³⁸ Dies garantiert, dass alte, fehlerhafte Restdaten ausradiert werden und der Backtest ausschließlich auf den verifizierten Dukascopy-Daten operiert.³⁸ Zudem wird durch CustomSymbolSetInteger der SYMBOL_CHART_MODE explizit auf SYMBOL_CHART_MODE_BID gesetzt, um Darstellungsfehler bei außerbörslichen Instrumenten zu vermeiden, die keine Last-Preise aufweisen.³⁵

3. GitHub Prior Art (Bestehender Code & Projekte)

Die Analyse bestehender Repositorien auf GitHub und in spezialisierten Entwickler-Foren offenbart wiederkehrende Muster und Architektur-Paradigmen in der Automatisierung von MT5-Infrastrukturen. Diese Erkenntnisse dienen als Fundament, um bekannte Fehlerquellen bei der Konzeption des Java-Systems zu vermeiden.

Evaluierung von Open-Source Architektur-Ansätzen (Prior Art)

Repository	Sprache	Kern-Fokus	Architektur-Paradigma
EA31337/EA-Tester	Shell / Python	MT Auto Login Testing & Befehlszeilen-Steuerung	CLI-Ausführung (Wine) mit detaillierter .ini Konfiguration
fortesenselabs/metatrader-terminal	Python	Server-Brücke für Automatisierung, Copy Trading & Backtesting	Docker (VNC-Alpine), Socket.io Server, Openbox minimal WM
giuse88/duka	Python	Hochgeschwindigkeits-Download von Dukascopy Tick-Daten	CLI-Terminalanwendung, Multi-Threading & Coroutines
nj4x (NuGet Package)	Java / .NET	Terminal-Schnittstelle für Mechanische Trading Systeme (MTS)	Hintergrund-Terminal-Server & direkter Socket Server
mayeranalytics/bi5	Rust	Parser für bi5 Dukascopy Tick-Dateien	Kommandozeilen-Dienstprogramm, Binär-Parsing (LZMA encoded)

Die Analyse zeigt eine klare Tendenz weg von direkten In-Memory Injektionen hin zu CLI-gesteuerten Konfigurationsarchitekturen und REST/Socket-basierten Kommunikationsprotokollen für MetaTrader 5.

Datenquellen: [metatrader-terminal](#), [duka](#), [nj4x](#), [mayeranalytics/bi5](#)

3.1 Automatisierung von Backtests

Zwei dominante Open-Source-Projekte im Bereich der MT5 CLI-Steuerung sind der **"EA-Tester"** und das Repository **"fortesenselabs/metatrader-terminal"**. Diese Projekte zielen darauf ab, MT5 in CI/CD-Pipelines zu integrieren. Sie setzen stark auf Containerisierung (Docker und Wine auf Linux-Derivaten wie Alpine), weisen jedoch strukturell den Weg für robuste Automatisierungsansätze. Ein wesentliches Architektur-Paradigma, das aus dem metatrader-terminal Repo hervorgeht, ist der strikte Verzicht auf das Schreiben von tester.ini-Dateien in systemweite oder geteilte Konfigurationsordner. Stattdessen wird die Terminal-Ausführung isoliert, indem der Parameter /portable genutzt wird und der Parameter /config: auf einen temporären, prozessexklusiven Pfad (z.B. /config:C:\TempRun123\tester.ini) verweist. Die Java-Applikation sollte diese Methodik der Isolation übernehmen: Pro Backtest-Auftrag wird ein temporärer Ordner für die INI-Datei und die zu erwartenden

XML-Berichte erzeugt. Dies schließt Race Conditions aus, wenn das System später erweitert wird, um parallele Backtests verschiedener Instrumente durchzuführen.⁵

Ebenfalls relevant für das Verständnis der Systemgrenzen ist das Projekt **StrategyTester5** (in Python geschrieben).¹⁶ Auch wenn dieses Projekt die MT5-Order-Logik in Python nachbaut, anstatt den nativen Tester zu nutzen, zeigt es ein wichtiges Pattern für das Management von Einstellungen: Die Konfiguration des Backtests (Magic Number, Slippage, Perioden, Startkapital) wird idealerweise in flexiblen JSON-Dateien gehalten und erst zur Laufzeit in das MT5-native INI-Format oder als Parameter konvertiert.¹⁶ Die Java-Lösung sollte analog dazu intern mit JSON-Profilen arbeiten und die INI-Generierung via Templating durchführen.

3.2 Parsing und Verarbeitung der Dukascopy Daten

Für den Prozess des Herunterladens und der Konvertierung von .bi5 Dateien gibt es hochspezialisierte Bibliotheken, die wertvolle Erkenntnisse liefern:

- **duka (Python):** Dieses Tool lädt Daten extrem performant via Threads und Coroutines.²⁵ Der Quellcode auf GitHub verdeutlicht das zugrundeliegende URL-Muster der Dukascopy-Server für den Abruf stündlicher Historien.²⁰
- **mayeranalytics/bi5 (Rust/C):** Ein hochleistungsfähiger Parser, der auf Low-Level-Sprachen setzt, um die binären Blöcke zu dekomprimieren und zu lesen.¹⁹ Er demonstriert die exakte Dekodierung der Big-Endian-Strukturen.¹⁹
- **ninety47/dukascopy (C++):** Ein weiteres Projekt, welches das LZMA-Dekompressionsverfahren und die anschließende Multiplikation der Floating-Point-Werte mit der Point-Size (z.B. 100.000) dokumentiert.²²

Wie in Kapitel 2 dargelegt, kann die Java-Architektur diese Logik nativ implementieren oder zur Entlastung ein etabliertes Skript aufrufen. Die Prior Art zeigt jedoch, dass die Konvertierung von LZMA zu CSV extrem CPU-intensiv ist. Die Auslagerung in einen separaten Prozess-Thread ist zwingend erforderlich.²¹

3.3 Kommerzielle und Heavy-Weight Alternativen

Das Projekt **NJ4X** stellt eine existierende, sehr umfangreiche Java-Brücke zum MT4 und MT5 Terminal dar.⁴³ Es betreibt einen lokalen Socket-Server, um Expert Advisors und Live-Trades direkt aus Java heraus zu orchestrieren, anstatt den Umweg über Dateien zu gehen.⁴³ Dieses Paradigma (dauerhafte Live-Socket-Verbindung) ist zwar mächtig für algorithmisches Live-Trading, für den Anwendungsfall eines automatisierten Backtesters jedoch überdimensioniert, schwergewichtig und wartungsintensiv. Die Analyse des Prior Arts bestätigt die Validität der hier vorgeschlagenen Architekturvorgabe: Eine zustandslose, dateibasierte Steuerung (INI-Datei als Input, XML-Report als Output) via CLI ist für *Testing*-Zwecke wesentlich stabiler und entkoppelter als komplexe In-Memory- oder Socket-Injektionen.

4. Architektur-Vorgaben für die Java GUI

Die Java-Applikation bildet das Kontrollzentrum der gesamten Operation. Die Kommunikation zwischen der GUI (beispielsweise implementiert in JavaFX für moderne, reaktionsschnelle asynchrone Oberflächen) und dem externen, in C++ kompilierten MT5-Terminal erfordert höchste Präzision im Bereich des Thread-Managements. Blockierende Aufrufe auf dem Main-UI-Thread (JavaFX Application Thread) sind strikt zu vermeiden.

4.1 Prozess-Management und der 64KB-Deadlock

Der asynchrone Start der terminal64.exe wird über die Standard-Java-Klasse `java.lang.ProcessBuilder` realisiert. Die Architekturvorgabe verlangt eine strikte und fehlerfreie Behandlung der Standard- (Stdout) und Fehlerstreams (Stderr) des generierten Sub-Prozesses.⁸

Auf Windows-Systemen stellt der Betriebssystem-Kernel standardmäßig einen Puffer von lediglich 64 Kilobyte für Daten-Pipes zwischen dem Hauptprozess und seinen Sub-Prozessen zur Verfügung.⁹ Wenn das MetaTrader-Terminal (selbst im Headless-Modus) Ausgaben, Warnungen oder Logs auf Stdout oder Stderr generiert und das Java-Programm diese Streams nicht aktiv und kontinuierlich ausliest, füllt sich dieser kleine Puffer rasant. Sobald die 64KB-Grenze erreicht ist, blockiert das Windows-Betriebssystem den MT5-Terminal-Prozess, da dieser seine Log-Ausgaben nicht mehr absetzen kann. Das führt zu einem fatalen Deadlock – die Java-Anweisung `process.waitFor()` wird in diesem Szenario niemals zurückkehren, und der Backtest bleibt hängen.⁸

Die Implementierung muss daher die Fehler- und Standardausgaben zusammenführen und zwingend in einem separaten, parallelen Thread konsumieren:

Java

```
import java.io.*;
```

```
public class MT5ProcessManager {
```

```
    public int runBacktest(String terminalPath, String iniPath) throws Exception {
        ProcessBuilder pb = new ProcessBuilder(
            terminalPath,
            "/portable",
            "/config:" + iniPath
        );
```

```
        // KRITISCH: Zusammenführung der Streams zur Verhinderung von Buffer-Overflows
        pb.redirectErrorStream(true);
```

```

    Process process = pb.start();

    // Asynchroner Stream-Consumer-Thread, der den 64KB Buffer permanent leert
    Thread outputConsumer = new Thread(() -> {
        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(process.getInputStream()))) {
            String line;
            while ((line = reader.readLine()) != null) {
                // Die Ausgabe kann in der Java-GUI in einer Konsole angezeigt
                // oder in ein Logfile geschrieben werden.
                System.out.println(" " + line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    });

    outputConsumer.setDaemon(true);
    outputConsumer.start();

    // Blockieren des Background-Tasks bis MT5 (ShutdownTerminal=1) sich regulär beendet
    int exitCode = process.waitFor();

    // Sicherstellen, dass der Consumer-Thread die letzten Zeilen verarbeitet hat
    outputConsumer.join(5000);

    return exitCode;
}

```

4.2 Parsing der MT5 XML-Ergebnisberichte

Sobald `waitFor()` einen Exit-Code von 0 (Success) zurückmeldet, liegt im spezifizierten Verzeichnis die Report-Datei des Strategietesters vor.¹ Der MT5-XML-Report ist eine strukturierte Abbildung der Testergebnisse. Eine Analyse der von MT5 generierten XML-Dateien zeigt, dass diese nicht als reines Daten-XML (wie REST-APIs), sondern oftmals als strukturiertes HTML gekapselt in XML-Tags formatiert sind.⁴⁵

Die Struktur beinhaltet tabellarische Datenknoten (z.B. `<report>`, `<test>`, `<tr>`, `<td>`), die Parameter und Leistungskennzahlen wie *Total Net profit*, *Profit Factor*, *Recovery Factor*, *Expected Payoff* und den *Maximal Drawdown* beinhalten.⁷

Für das robuste Deserialisieren dieser großen, potenziell verschachtelten XML-Dateien wird der Einsatz der Bibliothek **FasterXML Jackson Dataformat XML** dringend empfohlen.⁴⁶ Klassische

SAX oder DOM-Parser aus der Java Standardbibliothek sind in der Implementierung zu verbos, fehleranfällig gegenüber leichten Strukturänderungen und langsamer in der Erzeugung von nutzbaren Objekten.⁴⁹

Jackson erlaubt das direkte, deklarative Mappen der XML-Knoten auf *Plain Old Java Objects* (POJOs) mittels spezifischer Annotationen. Die Konfiguration des Jackson XmlMapper sollte dabei striktes Parsing deaktivieren. Dies ist essenziell, um bei Änderungen am MT5-Report-Format in zukünftigen Terminal-Updates (z.B. wenn MetaQuotes neue Metriken hinzufügt) keine Exceptions zu werfen (DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES auf false setzen).⁴⁸

Java

```
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlProperty;
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.dataformat.xml.XmlMapper;
import com.fasterxml.jackson.databind.DeserializationFeature;

@JsonIgnoreProperties(ignoreUnknown = true)
public class MT5Report {

    // Beispielhaftes Mapping, abhängig von der exakten XML-Struktur des aktuellen MT5 Builds
    @JacksonXmlProperty(localName = "TotalNetProfit")
    private double totalNetProfit;

    @JacksonXmlProperty(localName = "ProfitFactor")
    private double profitFactor;

    @JacksonXmlProperty(localName = "MaximalDrawdown")
    private double maximalDrawdown;

    // Getter und Setter...
}

// Initialisierung des Parsers in der Java-App
XmlMapper xmlMapper = new XmlMapper();
xmlMapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
// MT5Report report = xmlMapper.readValue(new File("report.xml"), MT5Report.class);
```

Da MetaTrader die Metriken teilweise in Tabellenzellen ohne semantische Key-Value-Tags

ausgibt⁴⁵, ist die Java-Applikation möglicherweise gezwungen, einen benutzerdefinierten Jackson-Deserializer (CustomDeserialzer) zu implementieren. Dieser iteriert über die Knoten und sucht nach Schlüsselwörtern wie "Total Net profit", um den numerischen Wert der benachbarten Zelle abzugreifen.⁴⁵

4.3 High-Performance Verarbeitung von Kursdaten (CSV)

Falls das Java-Programm die aggregierten CSV-Exporte der Dukascopy-Skripte noch veredeln muss – beispielsweise für die Anwendung eigener Zeitzonen-Shifts, das Filtern von Anomalien oder das Aggregieren von Tick-Daten zu höheren Timeframes – prallen enorme Datenmengen auf die Java Virtual Machine (JVM). Eine CSV-Datei mit 1-Minuten-Daten über 10 Jahre umfasst Millionen von Zeilen; Tick-Daten generieren schnell Gigabyte-große Dateien.

Performance-Benchmarks belegen, dass die Standard-String-Operationen in Java (wie `String.split()`) oder veraltete, schwergewichtige Bibliotheken wie `OpenCSV` bei der Verarbeitung solcher Finanzdaten extrem ineffizient sind und häufig zu *Out-of-Memory-Errors* (OOM) oder massiven Latenzen durch den Garbage Collector führen.⁵¹

Die Architekturvorgabe erfordert daher den zwingenden Einsatz von High-Performance-Parsing-Bibliotheken wie **uniVocity-parsers** oder **FastCSV**.⁵¹

- **FastCSV:** Diese Bibliothek besitzt absolut keine externen Laufzeitabhängigkeiten, arbeitet "Null-Free" und verarbeitet Daten ultraschnell, da sie unnötige Objekterzeugungen vermeidet und somit den Garbage Collector entlastet.⁵³
- **uniVocity-parsers:** Bietet die architektonisch wertvolle Funktion der Spalten-Selektion ("Column Selection").⁵¹ Dies erlaubt es, nur ausgewählte Spalten auszuwerten. Wenn das Java-System bei einer 8-Spalten-CSV (Time, Open, High, Low, Close, TickVol, RealVol, Spread) beispielsweise nur Close-Preise für Vorab-Analysen berechnen soll, ignoriert uniVocity die restlichen Spalten physisch während des Parse-Vorgangs. Dies reduziert den Speicherverbrauch und die Laufzeit drastisch.⁵¹

Zusammenfassung und Implementierungs-Roadmap

Die Schaffung einer Java-basierten MT5-Automatisierungslösung erfordert die strikte Isolation von Komponenten und ein tiefes Verständnis für die Limitationen von IPC (Inter-Process Communication). Der KI-Programmierer muss bei der Umsetzung der Software folgende architektonische Prinzipien verinnerlichen:

1. **Infrastruktur & Isolation:** Das Starten des MT5-Terminals erfolgt ausschließlich über `terminal64.exe /portable /config:....` INI-Dateien werden pro Testlauf dynamisch in isolierten Verzeichnissen generiert, um Race Conditions zu vermeiden.
2. **Thread-Sicherheit:** Verhinderung von Deadlocks im `ProcessBuilder` durch die zwingende, asynchrone Konsumierung von Output- und Error-Streams. Das Warten auf den Abschluss (`waitFor()`) geschieht in einem Background-Thread, niemals im UI-Thread.
3. **Datenaufbereitung & Zeitzonen:** Die Extraktion der LZMA-komprimierten

Dukascopy-Dateien erfordert zwingend eine Zeitzone-Korrektur (Verschiebung von UTC auf Broker-DST, z.B. EET), bevor die Daten genutzt werden können.

4. **MT5-Injektion:** Die Übergabe der konvertierten CSV-Daten an den MT5 erfolgt nicht manuell, sondern durch ein via Kommandozeile aufgerufenen MQL5-Skript. Dieses skriptbasierte Setup nutzt CustomSymbolCreate und CustomRatesReplace, um ein sauberes, lokales Test-Universum zu schaffen.
5. **Auswertung:** Das Parsing der resultierenden XML-Report-Strukturen erfolgt durch ausfallsicher konfigurierte Mapper (FasterXML Jackson), um die gewonnenen Leistungskennzahlen zuverlässig in der Java GUI zu visualisieren.

Durch die strikte Einhaltung dieses Architekturbildes lassen sich asynchrone, parallele und vor allem hoch-performante Strategie-Optimierungs-Cluster realisieren, welche die Limitierungen der nativen MetaTrader 5 Benutzeroberfläche weit überwinden.

Referenzen

1. Platform Start - For Advanced Users - Getting Started - MetaTrader 5 Help, Zugriff am April 5, 2026, https://www.metatrader5.com/en/terminal/help/start_advanced/start
2. Set Up MT4/MT5 on a Windows VPS (Step-by-Step) - HostStage, Zugriff am April 5, 2026, <https://www.host-stage.net/case-study/forex/mt4-mt5-vps-setup-guide/>
3. Running ex5 script from command line on Windows? - Forex Factory, Zugriff am April 5, 2026, <https://www.forexfactory.com/thread/1072981-running-ex5-script-from-command-line-on-windows>
4. README.md - fortesenselabs/metatrader-terminal - GitHub, Zugriff am April 5, 2026, <https://github.com/fortesenselabs/metatrader-terminal/blob/main/README.md>
5. Running test from script in 2022 with MT5 - Stack Overflow, Zugriff am April 5, 2026, <https://stackoverflow.com/questions/73766843/running-test-from-script-in-2022-with-mt5>
6. Export testing result in Open XML file by command line - Auto Trading - General - MQL5, Zugriff am April 5, 2026, <https://www.mql5.com/en/forum/328491>
7. Testing Report - Algorithmic Trading, Trading Robots - MetaTrader 5 Help, Zugriff am April 5, 2026, https://www.metatrader5.com/en/terminal/help/algotrading/testing_report
8. Process (Java Platform SE 8) - Oracle Help Center, Zugriff am April 5, 2026, <https://docs.oracle.com/javase/8/docs/api/java/lang/Process.html>
9. Java ProcessBuilder: Deadlocks, Zombies, and the 64KB Wall - Level Up Coding, Zugriff am April 5, 2026, <https://levelup.gitconnected.com/java-processbuilder-deadlocks-zombies-and-the-64kb-wall-8f754bc15bbc>
10. Java WatchService (with Examples) - HowToDoInJava, Zugriff am April 5, 2026, <https://howtodoinjava.com/java8/java-8-watchservice-api-tutorial/>

11. Watching files with WatchService - Java I/O - Waiting For Code, Zugriff am April 5, 2026,
<https://www.waitingforcode.com/java-i-o/watching-files-with-watchservice/read>
12. Watching Files With Java NIO - DZone, Zugriff am April 5, 2026,
<https://dzone.com/articles/listening-to-fileevents-with-java-nio>
13. How can I make my method wait for a file to exist before continuing - Stack Overflow, Zugriff am April 5, 2026,
<https://stackoverflow.com/questions/29472969/how-can-i-make-my-method-wait-for-a-file-to-exist-before-continuing>
14. MetaTrader 5 platform build 2085: Integration with Python and Strategy Tester improvements, Zugriff am April 5, 2026,
<https://www.metatrader5.com/en/releasenotes/terminal/2085>
15. Python Integration - MQL5 Reference, Zugriff am April 5, 2026,
https://www.mql5.com/en/docs/python_metatrader5
16. MegaJocTan/StrategyTester5: A strategy tester for MetaTrader5 in Python language - GitHub, Zugriff am April 5, 2026,
<https://github.com/MegaJocTan/StrategyTester5>
17. strategytester5 - PyPI, Zugriff am April 5, 2026,
<https://pypi.org/project/strategytester5/1.7.0/>
18. Python command to execute non-Python (MQL5) files? - Stack Overflow, Zugriff am April 5, 2026,
<https://stackoverflow.com/questions/66968258/python-command-to-execute-non-python-mql5-files>
19. GitHub - mayeranalytics/bi5: Parser and CLI utility for bi5 tick files, Zugriff am April 5, 2026, <https://github.com/mayeranalytics/bi5>
20. bi5 | Lime Mojito, Zugriff am April 5, 2026, <https://limemojito.com/tag/bi5/>
21. Convert Dukascopy binary data to text (dataframe) in R - Stack Overflow, Zugriff am April 5, 2026,
<https://stackoverflow.com/questions/73601872/convert-dukascopy-binary-data-to-text-dataframe-in-r>
22. Library for handling binary data from Dukascopy tick files. - GitHub, Zugriff am April 5, 2026, <https://github.com/ninety47/dukascopy>
23. Reading Dukascopy bi5 Tick History with the TradingData Stream Library for Java, Zugriff am April 5, 2026,
<https://limemojito.com/reading-dukascopy-bi5-tick-history-with-the-tradingdata-stream-library-for-java/>
24. dukascopy-python - PyPI, Zugriff am April 5, 2026,
<https://pypi.org/project/dukascopy-python/>
25. duka - Dukascopy data downloader - GitHub Pages, Zugriff am April 5, 2026,
<https://giuse88.github.io/duka/>
26. Java/Tukaani/src/org/tukaani/xz/LZMAInputStream.java - platform/external/lzma - Git at Google - Android GoogleSource, Zugriff am April 5, 2026,
<https://android.googlesource.com/platform/external/lzma/+1e977d7/Java/Tukaani/src/org/tukaani/xz/LZMAInputStream.java>
27. Changing time zone in MT4 to New York's close - MQL5, Zugriff am April 5, 2026,

- <https://www.mql5.com/en/forum/121929>
28. What is VT Markets' GMT offset or server time?, Zugriff am April 5, 2026, <https://get.vtmarkets.help/hc/en-us/articles/37317868198297-What-is-VT-Markets-GMT-offset-or-server-time>
 29. What timezone is MetaTrader set on by default? - KCM Trade, Zugriff am April 5, 2026, <https://www.kcmtrade.com/hc/content/what-timezone-is-metatrader-set-on-by-default>
 30. Timezones and offsets handling when fetching MetaTrader5 data via mt5 API, Zugriff am April 5, 2026, <https://stackoverflow.com/questions/79595025/timezones-and-offsets-handling-when-fetching-metatrader5-data-via-mt5-api>
 31. Handling the Time Zone Issue: Step by Step - StrategyQuant Forum Topic, Zugriff am April 5, 2026, <https://strategyquant.com/forum/topic/7377-handling-the-time-zone-issue-step-by-step/>
 32. MT5: Create custom symbol | MetaTrader4/5 user guide - Myforex, Zugriff am April 5, 2026, <https://myforex.com/en/mt5guide/create-customsymbol.html>
 33. Custom Symbols - MQL5 functions - MQL4 Reference, Zugriff am April 5, 2026, https://docs.mql4.com/mql5_language/mql5_functions/mql5_customsymbols
 34. Custom Financial Instruments - For Advanced Users - Trading Operations - MetaTrader 5, Zugriff am April 5, 2026, https://www.metatrader5.com/en/terminal/help/trading_advanced/custom_instruments
 35. CustomSymbolSetInteger - Custom Symbols - MQL5 Reference, Zugriff am April 5, 2026, <https://www.mql5.com/en/docs/customsymbols/customsymbolsetinteger>
 36. MetaTrader 5 Build 1640: creating and testing custom symbols - Release Notes, Zugriff am April 5, 2026, <https://www.metatrader5.com/en/releasenotes/terminal/1459>
 37. Need any example CSV for import - MT5 - MT5 - General - MQL5 programming forum, Zugriff am April 5, 2026, <https://www.mql5.com/en/forum/455073>
 38. Adding, replacing, and deleting quotes - Advanced language tools - MQL5, Zugriff am April 5, 2026, https://www.mql5.com/en/book/advanced/custom_symbols/custom_symbols_rates
 39. CustomSymbolCreate - Custom Symbols - MQL5 Reference, Zugriff am April 5, 2026, <https://www.mql5.com/en/docs/customsymbols/customsymbolcreate>
 40. CustomRatesReplace - Custom Symbols - MQL5 Reference, Zugriff am April 5, 2026, <https://www.mql5.com/en/docs/customsymbols/customratesreplace>
 41. How to import historical data into csv to symbol custom using CustomRatesUpdate? - MQL5, Zugriff am April 5, 2026, <https://www.mql5.com/en/forum/273127>
 42. Price type for building symbol charts - Trading automation - MQL5 Programming for Traders, Zugriff am April 5, 2026, https://www.mql5.com/en/book/automation/symbols/symbols_chart_mode

43. nj4x 2.9.3 - NuGet Gallery, Zugriff am April 5, 2026, <https://www.nuget.org/packages/nj4x>
44. Cannot run exe file with ProcessBuilder in Java - Stack Overflow, Zugriff am April 5, 2026, <https://stackoverflow.com/questions/9017740/cannot-run-exe-file-with-processbuilder-in-java>
45. Analyzing trading results using HTML reports - MQL5 Articles, Zugriff am April 5, 2026, <https://www.mql5.com/en/articles/5436>
46. Parse XML to Java Objects Using Jackson - DZone, Zugriff am April 5, 2026, <https://dzone.com/articles/parse-xml-to-java-objects-using-jackson>
47. FasterXML/jackson-dataformat-xml: Extension for Jackson JSON processor that adds support for serializing POJOs as XML (and deserializing from XML) as an alternative to JSON - GitHub, Zugriff am April 5, 2026, <https://github.com/FasterXML/jackson-dataformat-xml>
48. Jackson XML Mapper - Mincong Huang, Zugriff am April 5, 2026, <https://mincong.io/2019/03/19/jackson-xml-mapper/>
49. Which is the best library for XML parsing in java - Stack Overflow, Zugriff am April 5, 2026, <https://stackoverflow.com/questions/5059224/which-is-the-best-library-for-xml-parsing-in-java>
50. Solving the XML Problem with Jackson - Stackify, Zugriff am April 5, 2026, <https://stackify.com/java-xml-jackson/>
51. uniVocity/csv-parsers-comparison - GitHub, Zugriff am April 5, 2026, <https://github.com/uniVocity/csv-parsers-comparison>
52. Performance comparison - SimpleFlatMapper, Zugriff am April 5, 2026, <https://simpleflatmapper.org/12-csv-performance.html>
53. Welcome to FastCSV | FastCSV, Zugriff am April 5, 2026, <https://fastcsv.org/>
54. Csv Parser Performance comparaison extended : r/java - Reddit, Zugriff am April 5, 2026, https://www.reddit.com/r/java/comments/2k0vb4/csv_parser_performance_comparaison_extended/